

Appendix E

Definition of the CPRL Virtual Machine

E.1 Specification

CVM (CPRL Virtual Machine) is a hypothetical computer designed to simplify the code generation phase of a compiler for CPRL (the Compiler PProject Language). CVM has a stack architecture; i.e., most instructions either expect operands on the stack, place results on the stack, or both. Memory is organized into 8-bit bytes, and each byte is directly addressable. A word is a logical grouping of 4 consecutive bytes in memory. The address of a word is the address of its first (low) byte. Boolean values are represented in a single byte, character values use 2 bytes (Unicode Basic Multilingual Plane or Plane 0, code points from U+0000 to U+FFFF), and integer values use a word (four bytes).

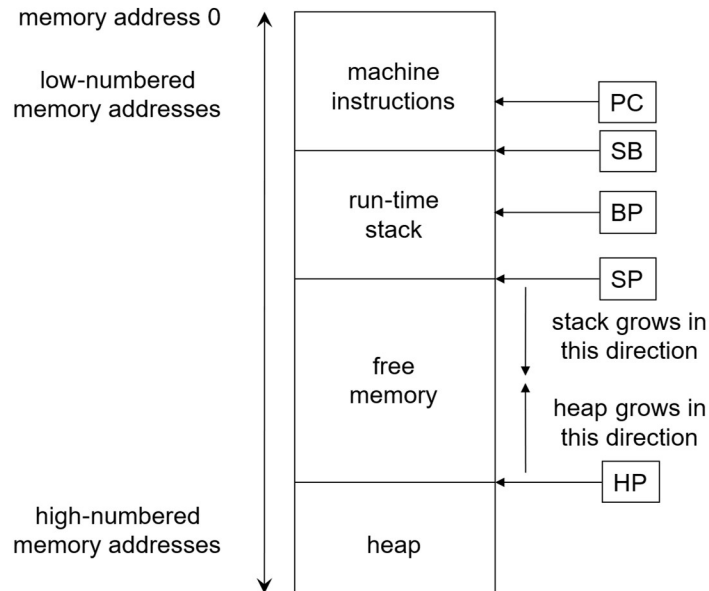
CVM has five 32-bit internal registers that are usually manipulated indirectly as a result of program execution. There are no general-purpose registers for computation. The names and functions of the internal registers are as follows.

- PC (program counter, a.k.a. instruction pointer): holds the address of the next instruction to be executed.
- SB (stack base): holds the address of the bottom of the stack. When a program is loaded, SB is initialized to the address of the first free byte in memory, and its value never changes during program execution.
- SP (stack pointer): holds the address of the top of the stack. The stack grows from low-numbered memory addresses to high-numbered memory addresses. When the stack is empty, SP has a value of $SB - 1$, the address immediately before the first free byte in memory.
- BP (base pointer): holds the base address of the current activation record (a.k.a. frame); i.e., the base address for the subprogram currently being executed. BP and SB have the same value (i.e., both reference the same memory location) if there are no currently active subprograms.
- HP (heap pointer): holds the address of the top of the heap. The heap grows from high-numbered memory addresses to low-numbered memory addresses. When a program is loaded, HP is initialized to the number of bytes in memory; i.e., one more than the last valid memory address. HP is used in only two CPRL string-related opcodes, GETSTR (reads a string from standard input) and STRCAT (concatenates two strings).

Each CVM instruction operation code (opcode) occupies one byte of memory. Some instructions take an immediate operand, which is always located immediately following the instruction in memory. Depending on the opcode, an immediate operand can be a single byte, two bytes (e.g., for a char), four bytes (e.g., for an integer or a memory address), or multiple bytes (e.g., for a string literal). The complete instruction set for CVM is given in the next section. Most instructions get their operands from the run-time stack. In general,

the operands are removed from the stack whenever the instruction is executed, and any results are left on the top of the stack. With respect to boolean values, zero means false and any nonzero value is interpreted as true.

The following diagram illustrates a program loaded into memory.



Variable Addressing

Each time that a subprogram is called, CVM saves the current value of BP and sets BP to point to the new activation record (a.k.a. frame). When the subprogram returns, CVM restores BP back to the saved value.

A variable has an absolute address in memory (the address of the first byte), but variables are more commonly addressed relative to a register. A local variable is addressed relative to register BP, and a global variable is addressed relative to register SB.

Various load and store operations move data between memory and the run-time stack.

E.2 Implementation

CVM is implemented by three classes in package `cvm`. Class `Constants` defines the number of bytes for primitive types plus the number of bytes for the context part of an activation record.

```
public static final int BYTES_PER_OPCODE = 1;
public static final int BYTES_PER_INTEGER = 4;
...
public static final int BYTES_PER_CONTEXT = 2*BYTES_PER_ADDRESS;
```

Class Opcode is an enum class that defines the name and numeric values for each CVM opcode. In addition, this class defines several helper functions used by the CVM, the assembler, and disassembler, including static method toOpcode(byte b) that returns the opcode for the specified byte value.

```
public enum Opcode
{
    // halt opcode
    HALT(0),

    // load opcodes (move data from memory to top of stack)
    LOAD(10),
    LOADB(11),
    LOAD2B(12),
    LOADW(13),
    LDCB(14),
    LDCCH(15),
    LDCINT(16),
    ...

    // arithmetic opcodes
    ADD(70),
    SUB(71),
    MUL(72),
    DIV(73),
    ...

    // program/procedure opcodes
    PROGRAM(90),
    PROC(91),
    CALL(92),
    RET(93),
    ...

    private final byte value;

    /**
     * Construct an opcode with its machine instruction value.
     */
    private Opcode(int value)
    {
        this.value = (byte) value;
    }

    ...
}
```

Class CVM is the primary component of the implementation for the virtual machine. In addition to several helper methods, every opcode is implemented by a method. CVM method `run()` provides the basic control logic for the virtual machine using a large “switch” statement to dispatch opcodes to their corresponding method calls.

```

public void run()
{
    running = true;
    pc = 0;

    try
    {
        while (running)
        {
            switch (Opcode.toOpcode(fetchByte()))
            {
                case ADD      -> add();
                case ALLOC    -> allocate();
                case BITAND   -> bitAnd();
                case BITOR    -> bitOr();
                case BITXOR   -> bitXor();
                case BITNOT   -> bitNot();
                case BR       -> branch();
                case BE       -> branchEqual();
                case BNE      -> branchNotEqual();
                case BG       -> branchGreater();
                case BGE      -> branchGreaterOrEqual();
                case BL       -> branchLess();
                ...
                case SHL      -> shl();
                case SHR      -> shr();
                case STORE     -> store();
                case STOREB    -> storeByte();
                case STORE2B   -> store2Bytes();
                case STOREW    -> storeWord();
                case STRCAT    -> strcat();
                case SUB       -> subtract();
                default        -> error("invalid machine instruction");
            }
        }
    }
    catch (Throwable t)
    {
        System.err.println("Virtual Machine Error: " + t.getMessage());
        t.printStackTrace();
    }
}

```

E.3 CVM Instruction Set Architecture

Mnemonic	Short Description	Stack before after	Definition
Arithmetic Opcodes			
ADD	Add: Pop two integers from the stack and push their sum back onto the stack.	n1 n2 n1 + n2	n2 ← popInt() n1 ← popInt() pushInt(n1 + n2)
SUB	Subtract: Pop two integers from the stack and push their difference back onto the stack.	n1 n2 n1 - n2	n2 ← popInt() n1 ← popInt() pushInt(n1 - n2)
MUL	Multiply: Pop two integers from the stack and push their product back onto the stack.	n1 n2 n1*n2	n2 ← popInt() n1 ← popInt() pushInt(n1*n2)
DIV	Divide: Pop two integers from the stack and push their quotient back onto the stack.	n1 n2 n1/n2	n2 ← popInt() n1 ← popInt() pushInt(n1/n2)
MOD	Modulo: Pop two integers from the stack, divide them, and push the remainder back onto the stack.	n1 n2 n1 % n2	n2 ← popInt() n1 ← popInt() pushInt(n1 % n2)
NEG	Negate: Pop an integer from the stack, negate it, and push the result back onto the stack.	n -n	n ← popInt() pushInt(-n)
INC	Increment: Pop an integer from the stack, add 1, and push the result back onto the stack.	n n + 1	n ← popInt() pushInt(n + 1)
DEC	Decrement: Pop an integer from the stack, subtract 1, and push the result back onto the stack.	n n - 1	n ← popInt() pushInt(n - 1)

Logical Opcodes			
NOT	Logical Not: Pop a byte from the stack and push its logical negation back onto the stack.	$\frac{b}{!b}$	<pre> b ← popByte() if b = 0 pushByte(1) else pushByte(0) </pre>
Shift Opcodes			
SHL	Shift Left: Pop 2 integers from the stack, shift the second integer left by the amount specified in the first integer using zero fill, and push the result back onto the stack. Note: Only the right most five bits of the first integer are used for the shift.	$\frac{n1 \quad n2}{n1 \ll n2}$	<pre> n2 ← popInt() n1 ← popInt() s ← n2 & 0b1111 pushInt(n1 << s) </pre>
SHR	Shift Right: Pop 2 integers from the stack, shift the second integer right by the amount specified in the first integer using sign extent, and push the result back onto the stack. Note: Only the right most five bits of the first integer are used for the shift.	$\frac{n1 \quad n2}{n1 \gg n2}$	<pre> n2 ← popInt() n1 ← popInt() s ← n2 & 0b1111 pushInt(n1 >> s) </pre>
Branch Opcodes			
BR displ	Branch: Branch unconditionally according to displacement argument (may be positive or negative).		<pre> pc ← pc + displ </pre>

BE displ	Branch Equal: Pop two integers from the stack and compare them. If they are equal, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 == n2 pc ← pc + displ </pre>
BNE displ	Branch Not Equal: Pop two integers from the stack and compare them. If they are not equal, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 != n2 pc ← pc + displ </pre>
BG displ	Branch Greater: Pop two integers from the stack and compare them. If the second integer is greater than the first, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 > n2 pc ← pc + displ </pre>
BGE displ	Branch Greater or Equal: Pop two integers from the stack and compare them. If the second integer is greater than or equal to the first, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 >= n2 pc ← pc + displ </pre>

BL displ	Branch Less: Pop two integers from the stack and compare them. If the second integer is less than the first, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 < n2 pc ← pc + displ </pre>
BLE displ	Branch Less or Equal: Pop two integers from the stack and compare them. If the second integer is less than or equal to the first, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{n1}{n2}$	<pre> n2 ← popInt() n1 ← popInt() if n1 <= n2 pc ← pc + displ </pre>
BZ displ	Branch if Zero: Pop one byte from the stack. If it is zero, then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{b}{}$	<pre> b ← popByte() if b = 0 pc ← pc + displ </pre>
BNZ displ	Branch if Nonzero: Pop one byte from the stack. If it is nonzero then branch according to displacement argument (may be positive or negative); otherwise continue with the next instruction.	$\frac{b}{}$	<pre> b ← popByte() if b ≠ 0 pc ← pc + displ </pre>
Load/Store Opcodes			
LOAD n	Load multiple bytes onto the stack: The number of bytes to move is part of the instruction. Pop an address from the stack and push n bytes starting at that address onto the stack.	$\frac{addr}{b1, b2, \dots, bn}$	<pre> addr ← popInt(); for i ← 0..n-1 loop pushByte(mem[addr + i]) </pre>

LOADB	Load Byte: Load (push) a single byte onto the stack. The address of the byte is obtained by popping it off the stack.	$\frac{\text{addr}}{b}$	$\begin{aligned} \text{addr} &\leftarrow \text{popInt}() \\ b &\leftarrow \text{mem}[\text{addr}] \\ \text{pushByte}(b) \end{aligned}$
LOAD2B	Load Two Bytes: Load (push) two consecutive bytes onto the stack. The address of the first byte is obtained by popping it off the stack.	$\frac{\text{addr}}{\begin{matrix} b0 \\ b1 \end{matrix}}$	$\begin{aligned} \text{addr} &\leftarrow \text{popInt}() \\ b0 &\leftarrow \text{mem}[\text{addr} + 0] \\ b1 &\leftarrow \text{mem}[\text{addr} + 1] \\ \text{pushByte}(b0) \\ \text{pushByte}(b1) \end{aligned}$
LOADW	Load Word: Load (push) a word (four consecutive bytes) onto the stack. The address of the word is obtained by popping it off the stack.	$\frac{\text{addr}}{w}$	$\begin{aligned} \text{addr} &\leftarrow \text{popInt}() \\ w &\leftarrow \text{getWord}(\text{addr}) \\ \text{pushInt}(w) \end{aligned}$
LDCB <i>b</i>	Load Constant Byte: Fetch the byte immediately following the opcode and push it onto the stack.	$\frac{\quad}{b}$	$\text{pushByte}(b)$
LDCB0	Load Constant Byte 0: Optimized version of LDCB 0.	$\frac{\quad}{0}$	$\text{pushByte}(0)$
LDCB1	Load Constant Byte 1: Optimized version of LDCB 1.	$\frac{\quad}{1}$	$\text{pushByte}(1)$
LDCCH <i>c</i>	Load Constant Character: Fetch the character immediately following the opcode and push it onto the stack.	$\frac{\quad}{c}$	$\text{pushChar}(c)$
LDCINT <i>n</i>	Load Constant Integer: Fetch the integer immediately following the opcode and push it onto the stack.	$\frac{\quad}{n}$	$\text{pushInt}(n)$
LDCINT0	Load Constant Integer 0: Optimized version of LDCINT 0.	$\frac{\quad}{0}$	$\text{pushInt}(0)$
LDCINT1	Load Constant Integer 1: Optimized version of LDCINT 1.	$\frac{\quad}{1}$	$\text{pushInt}(1)$

LDCSTR s	Load Constant String: The value of pc (which is the address of the constant string) is pushed onto the stack. Then pc is advanced so as to skip over the string.	$\frac{pc}{}$	// pc is address of string pushInt(pc) $len \leftarrow \text{getInt}(pc)$ $pc \leftarrow pc + 4 + len$
LDLADDR n	Load Local Address: Compute the absolute address of a local variable from its relative address n and push the absolute address onto the stack.	$\frac{bp + n}{}$	pushInt($bp + n$)
LDGADDR n	Load Global Address: Compute the absolute address of a global (program level) variable from its relative address n and push the absolute address onto the stack.	$\frac{sb + n}{}$	pushInt($sb + n$)
STORE n	Store n bytes: Remove n bytes from the stack followed by an absolute address and copy the n bytes to the location starting at the absolute address.	$\frac{\text{addr} \quad b1 \quad b2 \quad \dots \quad bn}{}$	$\text{addr} = \text{mem}[\text{sp}-n-3]$ for $i \leftarrow n-1..0$ $\text{mem}[\text{addr}+i] \leftarrow \text{popByte}()$ popInt() // remove addr
STOREB	Store Byte: Store a single byte at a specified memory location. The byte to be stored and the address where it is to be stored are popped from the stack.	$\frac{\text{addr} \quad b}{}$	$b \leftarrow \text{popByte}()$ $\text{addr} \leftarrow \text{popInt}()$ $\text{memory}[\text{addr}] \leftarrow b$
STORE2B	Store Two Bytes: Store two bytes at a specified memory location. The bytes to be stored and the address where they are to be stored are popped from the stack.	$\frac{\text{addr} \quad b0 \quad b1}{}$	$b1 \leftarrow \text{popByte}()$ $b0 \leftarrow \text{popByte}()$ $\text{addr} \leftarrow \text{popInt}()$ $\text{mem}[\text{addr} + 0] \leftarrow b0$ $\text{mem}[\text{addr} + 1] \leftarrow b1$
STOREW	Store Word: Store a word (4 bytes) at a specified memory location. The word to be stored and the address where it is to be stored are popped from the stack.	$\frac{\text{addr} \quad w}{}$	$w \leftarrow \text{popWord}()$ $\text{addr} \leftarrow \text{popInt}()$ putWord(w, addr)

Type Conversion Opcodes			
BYTE2INT	Byte to Integer: Pop a byte from the stack and push an int with the popped value as its low order (least significant) byte and zeros as the three high order bytes	$\frac{b}{n}$	$b \leftarrow \text{popByte}()$ $n \leftarrow [0b0, 0b0, 0b0, b]$ $\text{pushInt}(n)$
INT2BYTE	Integer to Byte: Pop an integer from the stack and push its low order (least significant) byte back onto the stack.	$\frac{n}{b}$	$n \leftarrow \text{popInt}()$ $\text{pushByte}(n[3])$
CHAR2INT	Char to Integer: Pop a character from the stack and push an int with the popped value as the two low order (least significant) bytes and zeros as the two high order bytes	$\frac{c}{n}$	$c \leftarrow \text{popChar}()$ $n \leftarrow \text{charToInt}(c)$ $\text{pushInt}(n)$
INT2CHAR	Integer to Byte: Pop an integer from the stack and push its low order (least significant) byte back onto the stack.	$\frac{n}{c}$	$n \leftarrow \text{popInt}()$ $c \leftarrow \text{intToChar}(n)$ $\text{pushChar}(c)$
Special Opcode for String Concatenation			
STRCAT	String concatenate: Concatenate two strings whose addresses are on the top of the stack. The concatenated string is stored on the heap. Register hp (address of the concatenated string) is pushed onto the stack.	$\frac{s1addr \quad s2addr}{hp}$	$\text{addr1} \leftarrow \text{popInt}()$ $\text{addr2} \leftarrow \text{popInt}()$ $\text{len1} \leftarrow \text{getInt}(\text{addr1})$ $\text{len2} \leftarrow \text{getInt}(\text{addr2})$ $\text{len} \leftarrow \text{len1} + \text{len2}$ $\text{copyStrToHeap}(\text{addr1})$ $\text{copyStrToHeap}(\text{addr2})$ $hp \leftarrow hp - 4$ $\text{putInt}(\text{len}, hp)$ $\text{pushInt}(hp)$

Bitwise Opcodes			
BITAND	Bitwise and: Pop two integers from the stack, perform bitwise and, and push the result back onto the stack.	$\frac{n1}{n2}$ $n1 \ \& \ n2$	$n2 \leftarrow \text{popInt}()$ $n1 \leftarrow \text{popInt}()$ $\text{pushInt}(n1 \ \& \ n2)$
BITOR	Bitwise or: Pop two integers from the stack, perform bitwise or, and push the result back onto the stack.	$\frac{n1}{n2}$ $n1 \ \ n2$	$n2 \leftarrow \text{popInt}()$ $n1 \leftarrow \text{popInt}()$ $\text{pushInt}(n1 \ \ n2)$
BITXOR	Bitwise xor: Pop two integers from the stack, perform bitwise xor, and push the result back onto the stack.	$\frac{n1}{n2}$ $n1 \ \wedge \ n2$	$n2 \leftarrow \text{popInt}()$ $n1 \leftarrow \text{popInt}()$ $\text{pushInt}(n1 \ \wedge \ n2)$
BITNOT	Bitwise not: Pop an integer from the stack, perform bitwise not, and push the result back onto the stack.	$\frac{n}{\sim n}$	$n \leftarrow \text{popInt}()$ $\text{pushInt}(\sim n)$
Program/Procedure Opcodes			
PROGRAM n	Program: Initialize base pointer and allocate space on the stack for the program's local variables.		$bp \leftarrow sb$ $sp \leftarrow bp + n - 1$
PROC n	Procedure: Allocate space on the stack for a subprogram's local variables.		$sp \leftarrow sp + n$
CALL $disp$	Call: Call a subprogram, pushing current values for BP and PC onto the stack.	$\frac{bp}{p}$	$\text{pushInt}(bp)$ $\text{pushInt}(pc)$ $bp \leftarrow sp - 7$ $pc \leftarrow pc + disp$

RET <i>n</i>	Return: Return from a subprogram, restoring the old value for BP plus space on stack previously allocated for the subprogram's local variables and parameters.		$bpSave \leftarrow bp$ $sp \leftarrow bpSave - n - 1$ $bp \leftarrow getInt(bpSave)$ $pc \leftarrow getInt(bpSave + 4)$
RET0	Optimized version of RET 0.		$bpSave \leftarrow bp$ $sp \leftarrow bpSave - 1$ $bp \leftarrow getInt(bpSave)$ $pc \leftarrow getInt(bpSave + 4)$
RET4	Optimized version of RET 4.		$bpSave \leftarrow bp$ $sp \leftarrow bpSave - 5$ $bp \leftarrow getInt(bpSave)$ $pc \leftarrow getInt(bpSave + 4)$
ALLOC <i>n</i>	Allocate: Allocate space on the stack for future use.		$sp \leftarrow sp + n$
HALT	Halt: Stop the virtual machine.		halt
I/O Opcodes			
GETBYTE	Get Byte: Read digits from standard input, convert them to a byte, and store the byte at the address on top of stack.	<u>addr</u>	$addr \leftarrow popInt()$ $b \leftarrow readByte()$ $putByte(b, addr)$
GETEOF	Get value of EOF flag: Push the value of the EOF flag onto the top of the stack.	<u>eofFlag</u>	$push(eofFlag)$
GETINT	Get Integer: Read digits from standard input, convert them to an integer, and store the integer at the address on top of stack.	<u>addr</u>	$addr \leftarrow popInt()$ $n \leftarrow readInt()$ $putInt(n, addr)$
GETCH	Get Character: Read character from standard input and store it at the address on top of stack.	<u>addr</u>	$addr \leftarrow popInt()$ $c \leftarrow readChar()$ $putChar(c, addr)$

GETSTR	Get String: Read string from standard input, store the string (length plus characters) on the heap, and write the heap pointer to the address on top of stack.	$\frac{\text{addr}}{\quad}$	<pre> addr ← popInt() s ← readStr() heapAlloc(4 + 2*s.length) copyStringToHeap(s) addr ← hp </pre>
PUTINT	Put Integer: Pop integer from top of the stack and write its value to standard output.	$\frac{n}{\quad}$	<pre> n ← popInt() writeInt(n) </pre>
PUTCH	Put Character: Pop character from top of stack and write its value to standard output.	$\frac{c}{\quad}$	<pre> c ← popChar() writeChar(c) </pre>
PUTSTR n	Put String: Write a string of n characters to standard output. The string (length plus characters) was previously pushed onto the stack.	$\frac{s}{\quad}$	<pre> nBytes ← 4 + 2*n addr ← sp - nBytes + 1 strLen ← getInt(addr) for i ← 0..strLen-1 loop writeChar(mem[2*i]) sp ← sp - n </pre>
PUTEOL	Put End-of-Line: Write a line terminator to standard output.		<pre> write(EOL) </pre>